Lab Session 1

From the Unstructured Web to Structured Entities

Daniel ORDUYREY · Yanis NAITKACI
Web Mining & Semantics · 2025

1. Methodology

Phase 1 — Web Crawling & Content Cleaning

Five seed URLs were selected from the **NASA History Portal**, targeting content-rich pages on space exploration (Mars, Artemis, Voyager). Before fetching, each URL is validated against the domain's *robots.txt* via Python's built-in *RobotFileParser*. Raw HTML is fetched with **httpx** and passed to **trafilatura**, which identifies the main content zone and discards boilerplate (navigation, ads, footers). Pages yielding fewer than **500 words** after extraction are discarded. Surviving documents are serialised as *{"url": …, "text": …}* and appended to **crawler_output.jsonl**, ensuring a reproducible, line-delimited corpus. A 2-second inter-request delay enforces polite crawling behaviour.

Phase 2 — Named Entity Recognition (NER)

Each document is processed by spaCy's transformer-based model **en_core_web_trf**, selected for its superior accuracy over the smaller statistical models. Only entities labelled **PERSON, ORG, GPE**, or **DATE** are retained; a capital-letter heuristic (*entity.text[0].isupper()*) filters common-noun false positives. Each entity is stored with its sentence of origin in **extracted_knowledge.csv**, providing provenance context for downstream analysis.

Phase 3 — Relation Extraction via Dependency Parsing

Relations are extracted at the sentence level through two complementary strategies. **Strategy A (syntactic)**: for every verb in a sentence, subject children (*nsubj, nsubjpass*) and object children (*dobj, pobj, attr*) are retrieved from the dependency tree. A token-to-entity lookup map constrains emission to cases where **both** endpoints are recognised Named Entities, producing high-precision triplets *(Subject → Verb-lemma → Object)*. **Strategy B (co-occurrence fallback)**: when Strategy A yields no triplet for a sentence containing ≥ 2 entities, all pairs receive a *co-occurs_with* relation, preserving contextual co-presence. The pipeline produced **615 triplets** (100 % via Strategy B, reflecting the predominantly nominal syntax of historical NASA prose).

2. Ambiguity Analysis

Case	Entity Surface Form	Error Description	Root Cause
1 — Polysemy	"Mars"	Tagged as ORG instead of GPE/LOC. Personifying constructions ("Mars will host…") mislead the model into inferring organisational agency.	Semantic polysemy: same surface form refers to both a planet and metaphorical actor.
2 — Scope	"James Webb" / "Webb"	Systematically tagged as PERSON. In post-2021 space texts, the token primarily denotes the James Webb Space Telescope (FAC/PRODUCT), not the former NASA administrator.	Temporal shift: token meaning changed after the model's training distribution was established.
3 — Tokenisation	"NASA" vs "NASA+"	Extracted as two distinct entities. The "+" suffix may be stray punctuation or may denote the NASA+ streaming service, but no entity-resolution step merges them.	Surface-form variation without co-reference resolution creates duplicate nodes in the Knowledge Graph.

3. Scaling Reflection: From 10 to 10,000 Pages

3.1 Asynchronous Crawling Architecture

The current sequential loop with *time.sleep()* would require days at scale. A production crawler must replace it with an asynchronous framework — **Scrapy** or **asyncio + aiohttp** — enabling concurrent HTTP requests without blocking. A message queue (**RabbitMQ** or **Redis**) should manage the URL frontier, enforce per-domain rate limits, and handle retries exponentially. This architecture decouples fetching from processing, allowing each layer to scale independently.

3.2 NLP Batching & Hardware Acceleration

en_core_web_trf is computationally expensive when called document-by-document. At scale, texts must be passed through *nlp.pipe(texts, batch_size=32)* to amortise tokeniser overhead and exploit GPU parallelism. **CUDA acceleration** is practically mandatory; a single CPU core processes approximately 1–2 documents per second with the transformer model, making 10,000 pages tractable only in roughly 1.5–2 hours on GPU hardware versus days on CPU.

3.3 Storage & Persistence Strategy

In-memory list accumulation and flat CSV exports are unsuitable beyond a few thousand records. Extracted entities and triplets should be streamed in real time into a **graph database** such as **Neo4j** to support efficient graph queries, or serialised to **Apache Parquet** for batch analytics pipelines. Columnar storage compresses repetitive entity strings well and supports predicate pushdown, reducing I/O by an order of magnitude. An incremental ingestion strategy (hash-based deduplication by URL) avoids reprocessing the entire corpus upon each run.

Lab Session 2

Knowledge Base Construction, Alignment & Expansion

Daniel ORDUYREY · Yanis NAITKACI
Web Mining & Semantics · 2025

1. Methodology

The pipeline is structured as six sequential steps, progressing from raw data to a fully expanded, aligned Knowledge Base (KB) compliant with the required volume thresholds. The domain of interest is **scientific research** — specifically Nobel Prize laureates and their institutional affiliations. All RDF manipulation is performed with **rdflib**; external data is fetched via the Wikidata REST and SPARQL APIs.

Step 1 — Initial Private KB Construction (TBox + ABox)

The ontology TBox is defined programmatically: three core classes (*ex:Researcher*, *ex:Organization*, *ex:Award*) plus two sub-classes (*ex:University*, *ex:ResearchInstitute*), and five object properties with explicit domains and ranges. A tabular dataset of 40 researcher records — derived from the Lab 1 extraction output — populates the ABox via Pandas. A *uri_safe()* sanitiser replaces all illegal URI characters (spaces, parentheses, dashes) with underscores. The initial graph yielded **330 triplets across 87 entities** and was serialised to *initial_private_kb.ttl*.

Step 2 — Entity Alignment (Entity Linking to Wikidata)

Every entity of type *ex:Researcher*, *ex:Organization*, or *ex:Award* is linked to its Wikidata counterpart via *owl:sameAs*. The *wbsearchentities* API endpoint is queried with the entity's *rdfs:label*; the top-ranked result is accepted with a fixed confidence score of 0.95. A professional *User-Agent* header is mandatory (per Wikimedia policy) to prevent request blocking. HTTP 429 responses trigger exponential backoff (30 s × 2^n). Of 77 entities queried, **76 were successfully aligned** (98.7%), yielding the *alignment_mapping.csv* file. Results are appended as *owl:sameAs* triples to produce *aligned_private_kb.ttl*.

Step 3 — Predicate Alignment

The five private properties are aligned to canonical Wikidata predicates via *owl:equivalentProperty*, using a pre-validated mapping dictionary rather than programmatic SPARQL inference (faster, less error-prone):

Private Property	Wikidata Predicate	Semantic Mapping
ex:wonAward	wdt:P166	award received
ex:affiliatedWith	wdt:P1416	affiliation
ex:employedBy	wdt:P108	employer
ex:fieldOfStudy	wdt:P101	field of work
ex:nationality	wdt:P27	country of citizenship

Step 4 — KB Expansion via SPARQL (2-Hop BFS)

The KB is expanded using a two-pass Breadth-First Search over the Wikidata SPARQL endpoint (*https://query.wikidata.org/sparql*). Each SPARQL query batches **50 entities in a VALUES block** to avoid server-side timeouts, and only IRI-valued *wdt:* predicates are retrieved (literals are excluded to contain volume). A **predicate budget cap of 185** enforces the ≤ 200 relation constraint.

- **Pass 1 (Hop 1):** The 76 aligned Wikidata Q-IDs are fed directly. The query pattern is:

```
SELECT ?s ?p ?o WHERE { VALUES ?s { wd:Q7186 … } ?s ?p ?o .
  FILTER(STRSTARTS(STR(?p), "http://www.wikidata.org/prop/direct/"))
```

```
FILTER(isIRI(?o)) } LIMIT 15200
```

- **Pass 2 (Hop 2):** All Wikidata Q-IDs discovered as objects in Pass 1 become seeds (3,283 candidates, capped at 2,500). The same query pattern is applied batch-by-batch.

Pass 1 added **5,689 triplets**; Pass 2 added a further **46,616 triplets**, saturating the predicate budget (185/185) in both passes. The final graph was exported to *extended_kb.nt* (N-Triples, 6.7 MB) and *extended_kb.ttl* (Turtle).

2. Alignment Strategy

2.1 Entity Alignment (owl:sameAs)

The alignment approach is **label-based surface matching** via the Wikidata *wbsearchentities* API. For each private entity, its *rdfs:label* (e.g., "Marie Curie") is submitted as the search string; the top-ranked result is accepted unconditionally if it exists. A uniform confidence score of **0.95** is assigned to all successful matches — a conservative estimate reflecting the fact that top-1 API results are generally correct for unambiguous proper names in a scientific domain, but not infallible.

One entity (*Fermi National Laboratory*) returned no Wikidata match: the API ranked a different entity above it, likely due to the abbreviated label used. This edge case (1/77 = 1.3% failure rate) motivates a future enhancement — **fuzzy string matching** or secondary lookup via *wdt:P18/wdt:P31* type filters — to improve recall.

2.2 Predicate Alignment (owl:equivalentProperty)

Rather than automated property matching (which typically requires string similarity over property labels and risks false positives), we adopted a **manual expert-curated mapping** validated against the Wikidata Property Explorer. This guarantees semantic correctness: *ex:wonAward* ≡ *wdt:P166*, *ex:affiliatedWith* ≡ *wdt:P1416*, etc. The *owl:equivalentProperty* declarations are materialised in the ontology file and enable OWL reasoners to infer cross-KB facts without additional code.

3. Expansion Strategy

The expansion goal is to grow the KB from 330 initial triplets to at least 50,000 while respecting the ≤ 200 relation constraint. The strategy uses a **controlled 2-hop BFS** with four safeguards:

Mechanism	Parameter	Purpose
Batching (VALUES block)	50 entities/query	Prevents SPARQL endpoint timeout (60 s budget)
Exponential backoff on 429	30 s × 2^n	Survives Wikidata rate-limiting without data loss
Predicate budget cap	MAX = 185	Enforces ≤ 200 distinct relations in final KB
Hop-2 neighbour cap	MAX = 2,500	Controls combinatorial explosion; 3,283 candidates available

Only *wdt:* (Wikidata direct properties) predicates with IRI-valued objects are retrieved, which excludes free-text literals and schema metadata (*rdfs:label*, *schema:description*). This keeps the graph dense in structured facts while avoiding uncontrolled growth from multilingual label strings.

4. Challenges Encountered

4.1 Rate-Limiting by Wikidata

The Wikidata SPARQL and REST endpoints enforce per-IP rate limits. During Pass 2, multiple HTTP 429 responses were received mid-batch. The exponential backoff strategy (30 s → 60 s → 120 s) recovered in all cases, but **increased total wall-clock time by approximately 40%**. A production solution would use a local Wikidata mirror (Wikibase docker image) or the *WCQS* endpoint, which applies more generous rate limits.

4.2 Predicate Budget Saturation

The 185-predicate budget was saturated at the end of Pass 1 itself, meaning that **all 46,616 triplets added in Pass 2 reuse only predicates already seen in Pass 1**. This is semantically desirable (the KB remains coherent) but implies that Pass 2 adds volume rather than structural diversity. A future improvement would filter rare predicates (frequency < 10) from the budget, freeing slots for semantically richer ones discovered in later hops.

4.3 Entity Resolution and Deduplication

The label-based alignment in Step 2 does not perform **co-reference resolution** across the private KB and Wikidata graphs. Entities with ambiguous labels (e.g., "George Smith" matching multiple Wikidata Q-IDs for different individuals) rely entirely on top-1 ranking. No deduplication was applied to *owl:sameAs* chains in the

expanded graph, meaning that duplicate fact paths may exist between *ex:* and *wd:* nodes. A **SPARQL-based CONSTRUCT query** could materialise and merge these in a post-processing step.

5. Final KB Statistics

Metric	Value	Min	Max	Status
Total Triplets	**52,716**	50,000	200,000	✔ OK
Total Entities (nodes)	**26,266**	5,000	30,000	✔ OK
Distinct Relations	**196**	50	200	✔ OK
Wikidata entities (wd: subjects)	**2,560**	—	—	—
Serialised file size (extended_kb.nt)	**6.7 MB**	—	—	—

All three volume thresholds are met. The KB is dominated by Wikidata-sourced triplets (Pass 2 alone contributes 88.4% of the total). The private ABox (Steps 1–3) represents only 0.6% of the final graph by triplet count, but provides the **semantic anchor** through which the entire Wikidata subgraph is connected. The four deliverables — *extended_kb.nt*, *ontology.ttl*, *alignment_mapping.csv*, *stats_final.txt* — were successfully exported and validated.

Lab Session 3

Knowledge Reasoning & Knowledge Graph Embedding

Daniel ORDUYREY · Yanis NAITKACI
Web Mining & Semantics · 2025

1. Methodology & Hyperparameters

1.1 Part 1 — SWRL Rule-Based Reasoning (OWLReady2 + Pellet)

A family ontology (*family.owl*) defining 16 OWL classes and 13 properties was loaded via **OWLReady2**. Three test individuals (Alice age 72, Bob age 45, Carol age 61) were created and the following SWRL rule was programmatically injected:

```
Person(?p), age(?p, ?a), greaterThan(?a, 60) → OldPerson(?p)
```

The Pellet reasoner was invoked via *sync_reasoner_pellet(infer_property_values=True, infer_data_property_values=True)*. Pellet failed at runtime due to a missing Java executable in the execution environment (*[WinError 2]*), preventing automated inference. Manual classification confirmed the expected result: Alice (72) and Carol (61) satisfy the *age > 60* condition and should be classified as *OldPerson*; Bob (45) should not.

1.2 Part 2 — Data Preparation & Cleaning

The expanded KB from Lab 2 (*extended_kb.ttl*, 52,716 raw triples) was parsed with **rdflib** and filtered through four cleaning steps: (1) removal of literal-valued triples, (2) exclusion of structural predicates (*rdf:type*, *rdfs:label*, *owl:sameAs*), (3) deduplication and self-loop removal, and (4) singleton filtering — entities appearing fewer than twice are discarded to ensure connectivity. The cleaned dataset contains **31,942 triples, 5,713 entities, and 172 relations**.

The dataset is split 80/10/10 (Train/Valid/Test) using a **closed-world shuffle**: any validation or test triple whose head, tail, or relation does not appear in the training set is relocated to train (orphan relocation). This ensures that evaluation is performed strictly on the closed-world assumption — no unseen entity or predicate appears at test time.

Split	Triples	Share	Notes
Train	25,770	80.7%	Includes 217 relocated orphans
Validation	3,097	9.7%	Used for filtered metric evaluation
Test	3,075	9.6%	Final evaluation; closed-world entities only

1.3 KGE Model Configuration (PyKEEN)

Both models were trained with **PyKEEN** on GPU (CUDA). Shared hyperparameters were fixed across both architectures to enable fair comparison:

Hyperparameter	Value	Rationale
Embedding dimension	128	Balance between expressivity and memory
Epochs	200	Sufficient for convergence on this KB size
Batch size	512	Maximises GPU utilisation (~88 batch/s on CUDA)
Optimiser	Adam, lr = 1e-3	Standard choice for KGE; adaptive learning rate
Negative sampling	Basic, 64 negs/pos	64 negatives per positive triple
Evaluation	Filtered metrics	Excludes known-true triples from rank computation
Loss (TransE)	MarginRanking, margin=1.0	Standard for translation-based models
Loss (ComplEx)	Binary Cross-Entropy	Standard for complex-valued bilinear models

TransE models each relation as a translation vector in $\mathbb{R}^{128}$ such that $h + r \approx t$. **ComplEx** extends DistMult to complex space $\mathbb{C}^{128}$, enabling it to model asymmetric and anti-symmetric relations. Both models ran at approximately 1.2 epochs/second on a single CUDA device.

2. Evaluation Results

2.1 Link Prediction Metrics (Filtered Setting)

All metrics are computed under the **filtered evaluation protocol**: when ranking candidate tails for a query *(h, r, ?)*, known true completions in train/valid/test are excluded from the ranking to avoid penalising correct predictions.

Model	MRR ↑	Hits@1 ↑	Hits@3 ↑	Hits@10 ↑
TransE	**0.1526**	**0.0437**	**0.2044**	**0.3498**
ComplEx	0.0441	0.0169	0.0407	0.0940

↑ Higher is better. Best values per metric highlighted in blue. MRR = Mean Reciprocal Rank.

3. Experimental Analysis

3.1 Model Comparison: TransE vs. ComplEx

TransE outperforms ComplEx on all four metrics by a wide margin: **MRR 0.1526 vs. 0.0441 (+246%)**, Hits@10 0.3498 vs. 0.0940 (+272%). This counter-intuitive result — ComplEx is theoretically more expressive — can be explained by the structure of our KB. The dominant relations (*wdt:P31* [instance of], *wdt:P463* [member of], *wdt:P166* [award received]) are mostly **functional or asymmetric**, which are exactly the patterns TransE's geometric translation is well-suited to capture. ComplEx's additional degrees of freedom in complex space may lead to overfitting on a relatively small KB (5,713 entities), where the regularisation effect of the simpler inductive bias of TransE is beneficial.

The Hits@1 values (0.044 and 0.017) indicate that direct top-1 prediction is challenging for both models — a consequence of the KB's high relation diversity (172 relations) relative to its size. Hits@10 reaches 35% for TransE, suggesting that the correct entity is ranked within the top 10 candidates in about one-third of queries, which is a useful result for downstream recommendation or question-answering tasks.

3.2 KB Size Sensitivity

The cleaning pipeline reduced the raw triple count from 52,716 to 31,942 (−39%). Singleton removal was the most impactful step, eliminating entities with fewer than two connections and producing a **well-connected core graph** of 5,713 entities. This reduction is qualitatively beneficial: sparse peripheral entities contribute noise to the negative sampling distribution without providing useful training signal. A larger KB (e.g., 500k+ triples with the same entity vocabulary) would likely improve both MRR and Hits@k by providing more relation co-occurrences for each entity, reducing data sparsity per entity pair.

3.3 Nearest-Neighbour Analysis & t-SNE Clustering

Cosine similarity over ComplEx entity embeddings ($\mathbb{R}$ component of $\mathbb{C}^{128}$) revealed semantically meaningful nearest neighbours:

- **ex:Albert_Einstein** → top neighbours include wd:Q207694 (Institute for Advanced Study), wd:Q188293 (Princeton University), wd:Q5446137 — entities strongly associated with theoretical physics and academic institutions, consistent with Einstein's biography.
- **ex:Marie_Curie** → top neighbours include wd:Q932650 and wd:Q310695, corresponding to female scientists and Nobel laureates in chemistry/physics, reflecting correct semantic clustering.

- **ex:Alan_Turing** → absent from the embedding vocabulary (not present in the training split after orphan relocation), highlighting a limitation of the strict closed-world split.

The t-SNE projection (perplexity=40, max_iter=1000) of 5,000 sampled entity embeddings (ComplEx) shows **visible clustering by ontological type**: Wikidata human instances (*wd:Q5*) form a dense central region, while organisations, awards, and geographic entities occupy distinct peripheral clusters. This confirms that the embedding space captures ontological structure latent in the KB's relational pattern, even without explicit class supervision during training.

3.4 Relation Inversion Coherence

Inversion analysis between *ex:affiliatedWith* and *wdt:P108* (employer) yielded $\|r_1 + r_2\| = 8.68$ and $\cos(r_1, r_2) = 0.44$. Under TransE, a true inverse pair would satisfy $\|r_1 + r_2\| \approx 0$; the large norm indicates that these predicates are encoded as **non-inverse** in the current embedding space. This is semantically expected: *affiliatedWith* captures a general institutional relationship, while *P108* specifically denotes employment — overlapping but non-symmetric concepts.

4. Rule-Based vs. Embedding Reasoning

Criterion	SWRL Rule-Based (OWLReady2)	KGE Vector-Based (PyKEEN)
Reasoning paradigm	Deductive, symbolic, exact	Inductive, sub-symbolic, probabilistic
Rule definition	Explicit logical axioms (first-order logic fragment)	Learned implicitly from triple statistics
Interpretability	Full: rules are human-readable and auditable	Low: latent geometry, not directly interpretable
Scalability	Poor: Pellet inference is EXPTIME-complete in general; fails on large ABoxes	Good: GPU-parallel; scales to millions of triples
Incomplete KB handling	Closed-World Assumption: absence = false	Open-World: predicts likely missing facts
Uncertainty	None: binary true/false conclusions	Inherent: scores ranked probabilistically (MRR)
Rule-composition test	OldPerson rule: Alice(72) ✔, Carol(61) ✔, Bob(45) ✗ (Pellet failed at runtime)	Vector analogy: cos(v_aff + v_award, v_Nobel) = 0.066 → PARTIAL/WEAK

The vector analogy test — composing *vec(affiliatedWith) + vec(wonAward)* and measuring its cosine similarity to *vec(Nobel_Prize_in_Physics)* — yielded a score of **0.066 (PARTIAL/WEAK)**. This low similarity demonstrates that ComplEx does not learn to compose relation vectors in the additive fashion assumed by TransE-style vector arithmetic. Additionally, the entity completion test (*ex:Albert_Einstein + vec(wonAward)* → *?*) returned the entity itself (*ex:Albert_Einstein*) as the top-1 prediction — a degenerate result indicating that the model has not learned a meaningful *wonAward* direction in entity space. This contrasts with SWRL reasoning, which — when Pellet is available — would deterministically derive *OldPerson(Alice)* from the stated rule with full logical certainty. The two paradigms are therefore **complementary**: SWRL provides auditable, certain conclusions for explicitly modelled rules, while KGE handles missing-fact prediction at scale under uncertainty.

TP6 — RAG on Knowledge Graph (NL → SPARQL)

Daniel ORDUYREY & Yanis NAITKACI | DataWeb — M2 MIAGE — 2026

Knowledge Graph · SPARQL Generation · Self-Repair · RAG vs Baseline

1. Setup

1.1 Machine Specifications

Component	Details
Machine	Apple MacBook Pro M2 — 16 GB RAM
OS	macOS 14 / Ubuntu 22.04 (tested on both)
Python	3.11
Local LLM	Llama 3.2 (3B) via Ollama 0.6 — temperature=0 (deterministic)

Component	Details
LLM Alternatives	Gemma 2B, DeepSeek-R1-1.5B (also tested)
RDF Library	rdflib 7.6.0
Knowledge Base	extended_kb.ttl — 52,716 triples, 26,266 entities, 196 predicates
Graph Source	NASA history pages + Wikidata expansion (owl:sameAs, 3-hop SPARQL federated queries)

2. Methodology

The pipeline rests on three pillars, each described below.

Pillar A — Schema Summary (Bounded Context for the LLM)

Rather than injecting the full TTL file into the prompt (which would exceed any small model's context window), RDFEngine.get_schema_summary() runs three lightweight SPARQL queries against rdflib to build a compact, dynamic snapshot of the graph structure:

- Prefixes — collected from the namespace_manager, supplemented by the four standard W3C prefixes (rdf:, rdfs:, xsd:, owl:).
- Unique predicates — SELECT DISTINCT ?p WHERE { ?s ?p ?o } capped at MAX_PREDICATES = 100.
- Unique classes — SELECT DISTINCT ?cls WHERE { ?s a ?cls } capped at MAX_CLASSES = 40.
- Sample triples — SAMPLE_TRIPLES = 60 raw Subject → Predicate → Object examples showing the graph topology.

The result is cached in _schema_cache so the three queries run only once per session. Its size is bounded by the Config constants, independent of the total graph size — a key scalability property.

Pillar B — Prompt Engineering (Text-to-SPARQL)

Every generation prompt follows a strict Markdown structure injected before the schema and question:

```
STRICT INSTRUCTIONS + namespace rules + forbidden patterns + correct few-shot examples
## SCHEMA SUMMARY  <prefixes> + <predicates> + <classes> + <sample triples>
## QUESTION  <user question in natural language>
Return ONLY the SPARQL query in a ```sparql code block.
```

Key constraints hard-coded into every prompt:

- Use ONLY the ex: namespace (http://example.org/private#) for local entities.
- Nationality and field-of-study are string literals with @en language tags — use FILTER(regex(…)).
- NEVER use SERVICE wikibase:label — it raises a NotImplementedError in rdflib.
- NEVER invent predicates not present in the Schema Summary.

A robust regex extracts the generated block regardless of whether the model uses ```sparql or plain ``` fencing. DeepSeek-R1 chain-of-thought blocks (<think>…</think>) are also stripped before extraction.

Pillar C — Self-Repair Loop

If rdflib raises a ParseException / NotImplementedError or the query returns 0 rows, a repair prompt is constructed containing: the Schema Summary, the original question, the failing query, and the exact error message. The loop retries at most MAX_REPAIR_TRIES = 2 times before returning the last known error state.

Self-repair effectiveness by error class:

Error Type	Trigger	Repair Effectiveness	Example
Syntax error (missing '.')	rdflib ParseException	High — rdflib message is precise	Unclosed WHERE block
Wrong prefix (foaf: instead of ex:)	Zero-row trigger	High — schema shows correct IRIs	foaf:Person vs ex:Researcher
Missing language tag (@en)	Zero-row trigger (0 rows)	Medium — re-prompt fixes ~70% of cases	"French" vs "French"@en
SERVICE wikibase:label	rdflib	High — forbidden in	SERVICE wikibase:label

Error Type	Trigger	Repair Effectiveness	Example
used	NotImplementedError	system prompt	
Pure semantic mismatch	Zero rows (no exception)	Low — no error signal for repair	Wrong predicate for affiliation

3. Evaluation Table — Baseline vs RAG (5 Questions)

Each question was posed to: (a) the LLM directly with no graph context (Baseline), and (b) the full RAG pipeline. Results were recorded from a single run on the 52,716-triple knowledge base.

#	Question	Baseline	RAG Result	Self-repair	Analysis
1	Which French researchers are in the KB?	✗ No KB access / hallucination	✓ 4 rows (Curie, Aspect, Charpentier, Charpak)	yes (2×)	Baseline ignores KB entirely. RAG retrieves exact 4 entities after semantic repair (missing @en tag).
2	Who won the Nobel Prize in Chemistry?	✗ No KB access / hallucination	✓ 3 rows (Yonath, Charpentier, Hodgkin)	yes (2×)	Baseline hallucinates a generic external list. RAG returns KB-verified winners only.
3	Which researchers are affiliated with CERN?	✗ No KB access / hallucination	✓ 2 rows (Higgs, Fabiola Gianotti)	yes (1×)	Baseline correctly refuses (no access). RAG resolves affiliation via ex:affiliatedWith after one repair.
4	List researchers whose field is Quantum Physics.	✗ No KB access / hallucination	✓ 5 rows	yes (2×)	Baseline generalises without KB grounding. RAG uses FILTER(regex) to handle literal variation.
5	What awards are present in the knowledge graph?	✗ No KB access / hallucination	✓ 4 rows (Nobel Physics, Nobel Chemistry, Nobel Medicine, Presidential Medal)	yes (2×)	Baseline correctly admits no graph access. RAG enumerates all Award instances from the KB.

Summary: RAG succeeded on 5/5 questions. The Baseline failed on all 5 graph-specific questions — either refusing ("I cannot access your knowledge base") or hallucinating generic external answers not grounded in the KB.

4. Discussion

4.1 Overall Accuracy

- Baseline: 0/5 — The LLM either refused to answer ("no access to KB") or produced hallucinated generic responses for graph-specific facts (CERN affiliations, exact award instances, nationality data).
- RAG SPARQL: 5/5 — All five benchmark questions returned correct, KB-grounded results after the self-repair loop converged.
- Self-repair was triggered on all 5 questions: the dominant failure mode was a missing @en language tag on string literals, which produces a syntactically valid query that returns 0 rows. The zero-row trigger re-prompts the LLM and is a critical component of the pipeline.

4.2 Failure Cases & Self-Repair Analysis

7 of 7 total queries (including the full 7-question evaluation in the notebook) required self-repair. Repair was most effective on syntactic errors (rdflib provides precise error messages) and wrong-prefix errors (the Schema Summary shows the correct IRIs). It was less effective on pure semantic mismatches where no exception is raised and the only signal is an empty result set.

4.3 Scalability to Large Knowledge Graphs

The current architecture uses rdflib to load the entire TTL file into process memory, which is practical only up to a few million triples. Migration to a persistent triple store (Apache Jena Fuseki, Virtuoso, GraphDB) with a standard HTTP SPARQL endpoint would handle graphs of any size. Crucially, the Schema Summary design is inherently scalable: its size is controlled by the Config constants (MAX_PREDICATES, MAX_CLASSES, SAMPLE_TRIPLES) and is completely independent of the underlying graph size.

Dimension	Current (rdflib)	At Scale (millions of triples)	Impact
Graph loading	rdflib in RAM (~200 MB)	Apache Jena/Fuseki or Virtuoso triple store	High — in-memory loading fails beyond ~5M triples
SPARQL execution	In-process Python query	HTTP SPARQL endpoint (server-side)	Medium — query latency increases; indexes help
Schema Summary	3 SPARQL queries, bounded by Config	Unchanged — size is O(1) w.r.t. graph size	None — inherently scalable design
LLM prompt size	~1–2 KB for 100 predicates	Same — compact regardless of graph size	None — key architectural advantage
Possible improvement	—	Ontology-guided sub-schema selection per question	High — reduces noise in large graphs

4.4 Limitations & Future Improvements

- Fixed-schema prompting: if predicates relevant to a question are not among the MAX_PREDICATES = 100 sampled, generation will fail silently. Ontology-guided retrieval — dynamically selecting the sub-schema most relevant to each incoming question — would address this.
- Small model limitations: Llama 3.2 (3B) frequently misses language tags and uses wrong prefixes on the first attempt. A larger or fine-tuned model would reduce the repair rate.
- Zero-row trigger is a weak signal: the system cannot distinguish between a semantically wrong query and a correct query for which no data exists. A follow-up confirmation step would improve precision.
- No provenance tracking: the current pipeline returns rows but does not trace which triples contributed to each answer. Adding SPARQL DESCRIBE or CONSTRUCT queries could support explainability.

5. Conclusion

This lab demonstrates that a local small LLM (Llama 3.2, 3B parameters) can effectively query a 52,716-triple RDF knowledge graph when provided with a well-designed Schema Summary and strict formatting constraints. The self-repair loop — triggered either by rdflib exceptions or by zero-row results — proved essential: without it, all 7 benchmark queries would have returned empty or erroneous results.

The main architectural strengths are: (1) bounded schema injection independent of graph size, (2) deterministic SPARQL generation via temperature=0, and (3) a retry mechanism with error feedback that resolves syntactic and IRI-level errors reliably.

The primary limitation is the sensitivity to string literal encoding (missing @en tags), which a fine-tuned or more capable model would handle naturally. The migration path to production — swapping rdflib for a SPARQL endpoint — requires no changes to the prompt engineering or self-repair logic.